

Nikhil Savant

312-989-8206 | nsavant033@gmail.com | [linkedin.com/in/nikhil-savant](https://www.linkedin.com/in/nikhil-savant) | U.S. Citizen

EDUCATION

University of Illinois Urbana-Champaign

Bachelor of Science in Computer Science and Economics

Urbana-Champaign, IL

Expected Graduation: 2028

Coursework: Database Systems, Algorithms and Models of Computation, Computer Systems, Data Structures

TECHNICAL SKILLS AND CLUBS

Languages: Java, Python, Go, TypeScript, C++, C, JavaScript, R, HTML/CSS, Swift, SQL

Frameworks: React, Node.js, Lit, Spring Boot, NextJS, React Native, LangChain

Developer Tools: Linux, CoreML, AWS, Jupyter, Git, MongoDB, Docker, Figma, Ansible, Nginx, Kubernetes, ChromaDB

Libraries: Pytorch, Pandas, OpenCV, NumPy, SK-Learn

Clubs: UIUC ACM Math and Algorithms Member, Engineering Council Activities Coordinator

WORK

Cisco

Software Engineer Intern

May - Aug 2026

San Jose, CA

- Built an AI agent over 9 live infrastructure tools via a TypeScript MCP gateway wrapping internal REST APIs as JSON-Schema-validated tools, standardizing discovery, argument validation, typed responses, and error handling
- Implemented delegated authorization using signed JWT claims and controller-level RBAC, validating and propagating caller identity/scopes before each MCP tool execution and preventing privilege escalation
- Engineered a repeatable RAG ingestion pipeline, embedding **2,300+** documents into **13,000+** vector-indexed chunks for semantic retrieval with deterministic source citation
- Built a 100-case agent evaluation suite measuring tool-selection accuracy, argument validity, retrieval Recall@K, citation correctness, and task success rate, improving end-to-end pass rate from **74% to 88%**

Algo Analytics

Full-Stack Engineer Intern

Jun - Oct 2025

Remote

- Developed a TypeScript-based backend integration layer to connect the React Native client with financial market APIs, aggregating real-time stock data and reducing client-side parsing times by **30%**
- Implemented REST API controllers to validate and serialize market data, processing **100,000+** data points daily while achieving a **300ms** API response time and trimming JSON payloads by **20%** by pruning unused response fields
- Added JWT Authentication with silent token refresh through client-side Axios interceptors, eliminating re-logins during active sessions

StellarPay

Software Engineer Intern

Jun - Aug 2025

Remote

- Built a GPT-powered financial assistant in TypeScript/Node.js using RAG, ingesting documents with AWS Textract and indexing OpenAI embeddings in Pinecone to raise retrieval precision by **40%**
- Refactored front-end components into shared modules, leading to a reduction of code duplication by **25%** and in page load times by **20%**
- Secured sensitive user data with IAM role-based access control, KMS-encrypted storage (AES-256), and JWT authorization on API Gateway

PROJECTS

Spotify Music Recommender

- Built a personalized music recommendation system that learns from 52M public listening events and creates a new Spotify playlist with the recommendations
- Developed an ETL pipeline that aggregates raw listening data into a user-item interaction matrix and resolves inconsistent track metadata across data sources
- Applied collaborative filtering through matrix factorization (ALS) with hyperparameter optimization and ranking-based evaluation, ensembling it with an item-similarity model to raise top-10 ranking quality from **0.30 to 0.71**
- Modeled shifting user taste by detecting recent listening trends and weighting them against long-term preferences, raising NDCG@100 from **0.05 to 0.11** on the user's newest additions

Container Runtime

- Developed a custom Go container runtime using Linux namespaces (PID, network, mount), capability restrictions, and seccomp filtering to isolate and execute application workloads
- Enforced CPU, memory, and process limits with cgroup v2, validating 64 concurrent workloads across 1,280 successful test requests with **2.06ms** p95 response latency on a 4-vCPU Linux VM
- Implemented shared OverlayFS image layers, achieving **5.16 ms** p95 warm startup and reducing allocated layer storage by **98.3%** versus duplicated base images across 64 instances